FMCG OrderHub – Technical Design Document (TDD)

Project: FMCG OrderHub
Selected Problem: FMCG Distributor's Retailer Ordering App with AI Order Capture
GitHub Repository: https://github.com/Thanusiyaks/Retailer-Order-Platform
Live Application: Not deployed yet – currently runs locally.

1. Project Overview

FMCG OrderHub is a web-based ordering platform designed for an FMCG distributor that supplies packaged food and home care products to kirana stores and other retailers.
The application aims to replace manual order collection through notebooks and WhatsApp messages with a centralized digital ordering system.
Retailers can log in, view products, check prices and stock availability, add products to a shopping cart, place orders, and track their order history.
Staff members can access orders from all retailers and export order information in CSV format for further processing in Tally.
The application also includes a rule-based text order capture feature that identifies supported product names and quantities from retailer messages.
The system is developed using React, Python, FastAPI, SQLAlchemy, and SQLite.

2. Problem Statement

The distributor currently receives retailer orders through handwritten notes, WhatsApp text messages, photos, and voice messages.
Office employees manually enter these orders into Tally, creating delays and increasing the possibility of data-entry errors.
Sales representatives also face difficulties checking product availability and applicable schemes while visiting retailers.
FMCG OrderHub addresses these challenges through:

- Centralized digital product catalogue.
- Retailer login and ordering.
- Product stock validation.
- Automated promotional scheme calculations.
- Rule-based text order capture.
- Retailer-specific order history.
- Staff order management.
- CSV export for Tally processing.

3. Objectives

The main objectives are:

1. Provide a simple digital ordering platform for retailers.
2. Display product prices, stock availability, and applicable schemes.
3. Allow retailers to create and track orders.
4. Identify supported products and quantities from text messages.
5. Validate stock before accepting orders.
6. Calculate applicable discounts and free quantities.
7. Provide staff with access to orders from all retailers.
8. Generate CSV files for further processing in Tally.

9. Protect retailer information through authentication and role-based access control.

4. Assumptions

The following assumptions were used to implement the demonstration application because the client brief does not specify every implementation detail.

Area	Assumption
Sample retailers	Demonstration retailer accounts are used for testing.
Sample products	Demonstration products and prices are configured in the database.
Order volume	The brief's assumption of 600 orders per day and 300 operating days is treated as a business context, not as a tested production capacity.
User roles	Two roles are implemented: Retailer and Staff.
Order confirmation	Orders are created with a Pending status.
Pricing	The backend calculates the final amount using configured product prices and schemes.
Product schemes	Example promotional schemes are configured for demonstration products.
Inventory	Product stock is maintained in the application database.
Order capture	The initial implementation uses predefined text-matching rules for supported products.
Tally processing	CSV export is used instead of direct Tally integration.
Deployment	The application is currently demonstrated in a local development environment.

These assumptions are implementation choices for the demonstration and are not presented as confirmed client requirements.

5. Technology Stack and Design Choices

5.1 Frontend

Technology	Purpose
React.js	Building reusable user interface components
JavaScript	Frontend application logic
HTML	Page structure
CSS	Styling and layout
Vite	Frontend development and production build

Reason for selection:

React supports reusable components and interactive interfaces. Vite provides a development environment and production build process.

5.2 Backend

Technology	Purpose
Python	Backend programming language
FastAPI	Building REST API endpoints
SQLAlchemy	Database ORM and query management
SQLite	Relational database
Uvicorn	Running the FastAPI application

Reason for selection:
FastAPI provides structured API development and request validation. SQLAlchemy simplifies database operations, while SQLite provides lightweight local data storage for the demonstration.

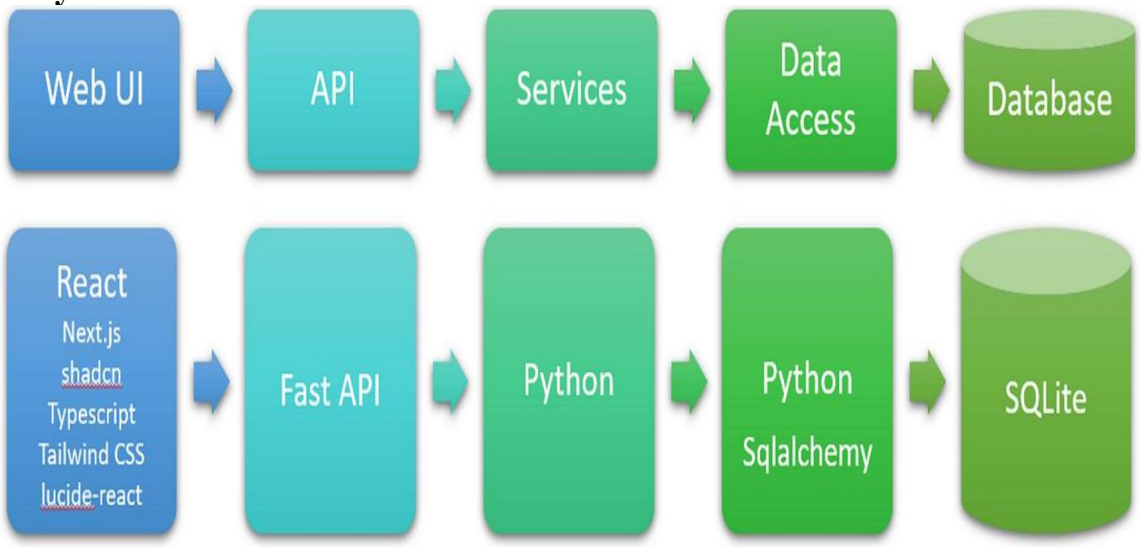
5.3 Authentication and Security

- PBKDF2-HMAC-SHA256 password hashing.
- HMAC-SHA256 signed authentication tokens.
- Eight-hour token expiration.
- Role-based access control.
- Protected API endpoints.
- Backend-side input and permission validation.

5.4 Development Tools

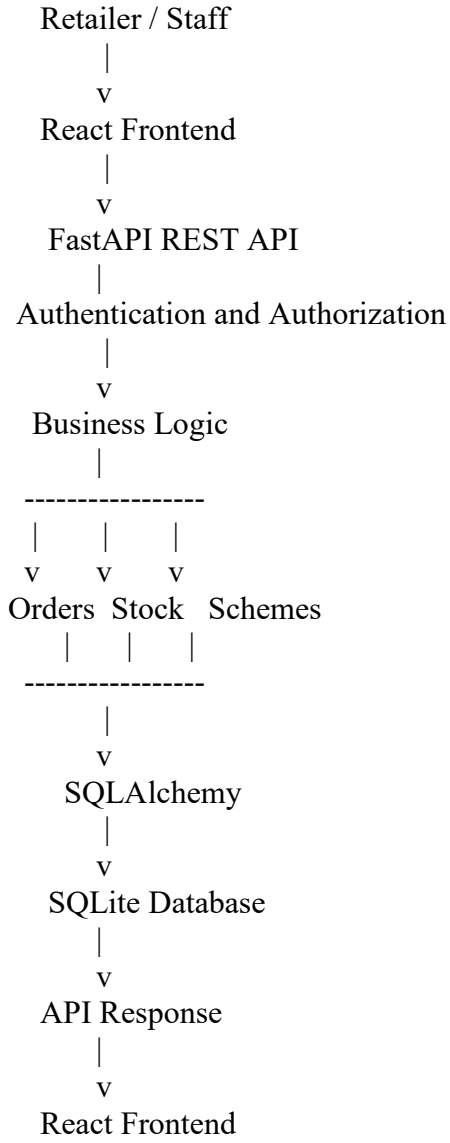
- Visual Studio Code.
- Swagger UI for API documentation and testing.
- WPS Spreadsheet for CSV verification.
- Git and GitHub for source-code management.

6. System Architecture



FMCG OrderHub follows a client-server architecture.
The React frontend communicates with the FastAPI backend through HTTP requests.
The backend handles authentication, authorization, business logic, stock validation, promotional calculations, and database operations.
SQLAlchemy is used to communicate with the SQLite database.

6.1 Architecture Flow



6.2 Main Architecture Components

Frontend Layer
Provides the login interface, retailer dashboard, product catalogue, shopping cart, order capture interface, order history, and staff order management screens.

Backend Layer
Handles authentication, user roles, order creation, product validation, inventory updates, scheme calculations, and CSV generation.

Database Layer
Stores users, retailers, products, orders, and order-item information.

7. User Roles and Permissions

7.1 Retailer

Retailers can:

- Log in using their account credentials.

- View their dashboard.
- View available products.
- Check prices, stock, and applicable schemes.
- Add products to the shopping cart.
- Increase or decrease product quantities.
- Capture supported orders from text messages.
- Review detected products and quantities.
- Place orders.
- View their own order history.

Retailers cannot view other retailers' orders or use staff-only export functionality.

7.2 Staff

Staff members can:

- Log in using staff credentials.
- Access the staff order management screen.
- View retailer information.
- View orders from all retailers.
- Review product quantities and order totals.
- Export order information as a CSV file.

Staff permissions are enforced through backend authorization checks.

8. Main Application Modules

8.1 Authentication Module

The authentication module validates usernames and passwords.

Passwords are stored as PBKDF2-HMAC-SHA256 password hashes rather than plain text.

After successful authentication, the backend generates an HMAC-SHA256 signed access token.

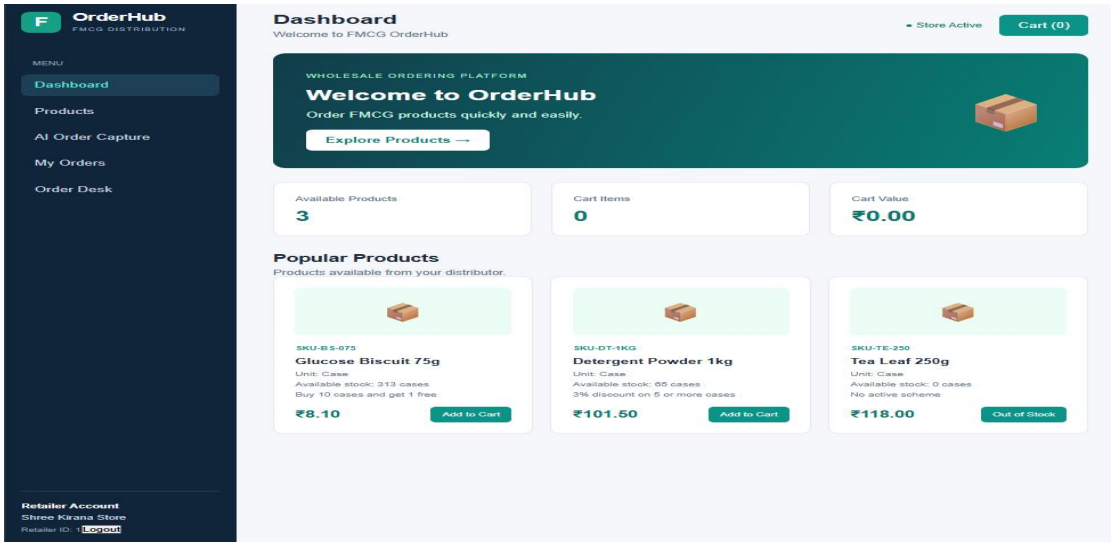
The token is used to access protected API endpoints and expires after eight hours.

8.2 Retailer Dashboard

The retailer dashboard provides an overview of the ordering application.

It provides access to product browsing and other retailer functions.

Retailer Dashboard



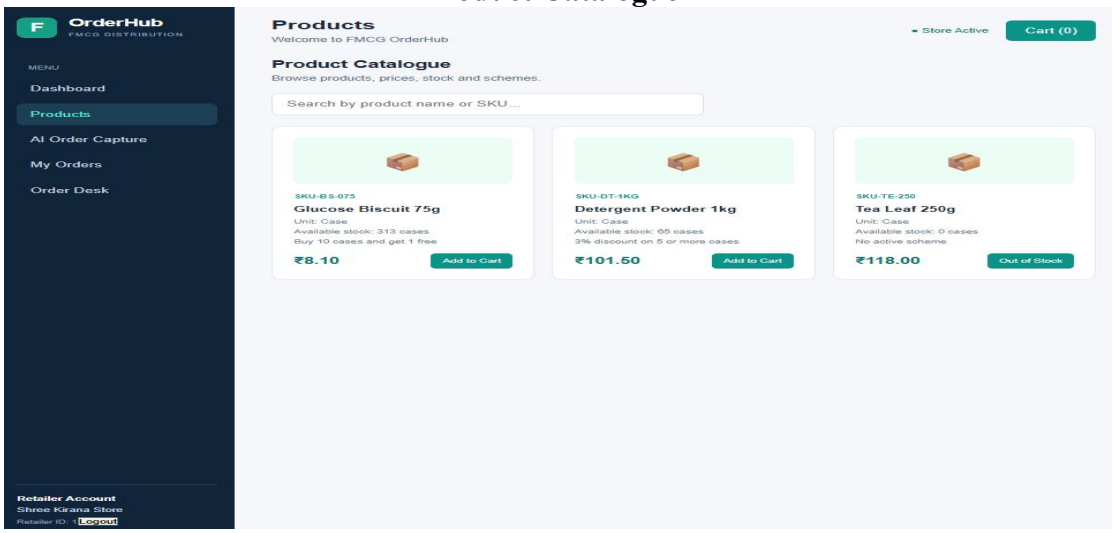
8.3 Product Catalogue Module

The product catalogue displays:

- Product name.
- SKU code.
- Dealer price.
- Available stock.
- Applicable schemes.

Retailers can browse products and add available products to their shopping cart. Products with zero available stock cannot be ordered.

Product Catalogue



8.4 Shopping Cart Module

The shopping cart allows retailers to:

- Add products.
- Increase or decrease quantities.
- Remove products.
- View estimated totals.
- Submit orders.

The frontend displays estimated totals. The backend performs the final pricing, scheme, and stock validation when an order is submitted.

Shopping Cart



8.5 Order Capture Module

The order capture module uses predefined, rule-based text matching to identify supported product names and quantities from retailer messages.
Examples of supported product references include:

- Biscuit.
- Detergent or washing powder.
- Tea or chai patti.

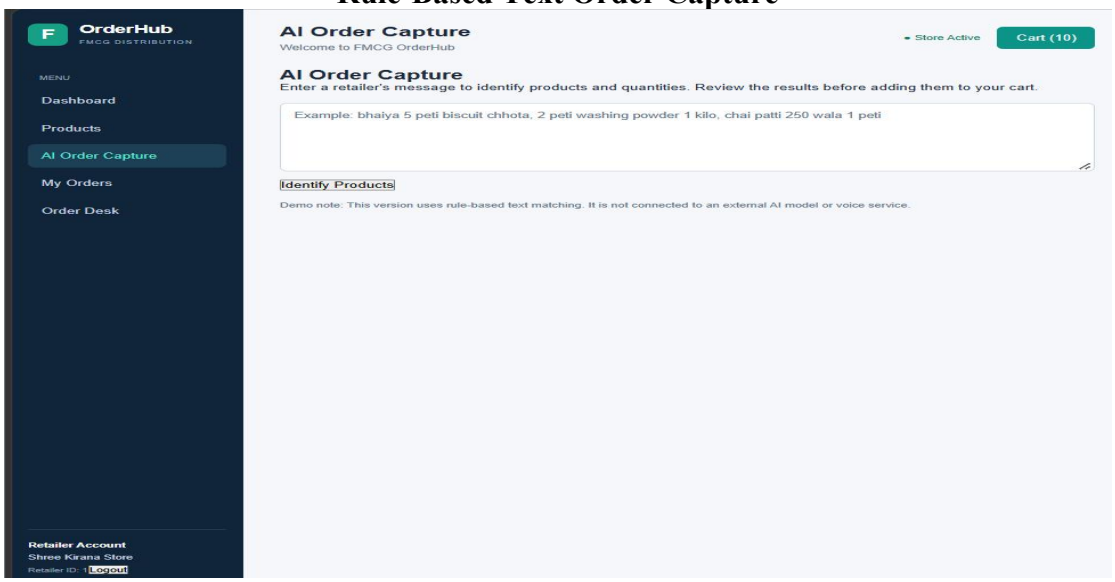
The system identifies supported product names and quantities from text input and displays the detected items for review before adding them to the shopping cart.

Handling incorrect or incomplete input:

- Unknown products generate a warning.
- Missing quantities generate a warning.
- Out-of-stock products generate a warning.
- The confirmation action is disabled when the input contains unsupported or incomplete product information.

This implementation uses predefined matching rules rather than an external AI model. Image-based order extraction and voice-note transcription are not currently implemented.

Rule-Based Text Order Capture



8.6 Order Management Module

When a retailer submits an order, the backend:

1. Validates the authenticated user.
2. Checks retailer information.
3. Validates the submitted products and quantities.
4. Rejects duplicate product entries.
5. Checks available product stock.
6. Calculates applicable schemes.
7. Calculates the final order amount.
8. Saves the order and order-item details.
9. Updates product stock.
10. Returns an order confirmation.

Retailers can view only their own orders. Staff can view orders from all retailers.

8.7 Pricing and Scheme Module

The application supports two example promotional schemes.

Buy X Get Y Free

For Glucose Biscuit 75g, purchasing 10 cases provides 1 additional free case.
The backend calculates the free quantity using the configured scheme value.

Percentage Discount

For Detergent Powder 1kg, orders of 5 or more cases receive a 3% discount.
The backend calculates the applicable discount and final payable amount.

8.8 Staff Order Management Module

The staff order management screen provides a centralized view of retailer orders.
Staff can review retailer names, order details, quantities, amounts, and order status.



8.9 Tally CSV Export Module

The staff-only export module generates a CSV file containing:

- Order ID.
- Date.
- Retailer name.
- Product name.
- Quantity.
- Free quantity.
- Unit price.
- Subtotal.
- Total amount.
- Order status.

Staff can download the CSV file and use it for further processing in Tally.
Direct Tally integration is not currently implemented.

9. Database Design

The application uses SQLite for persistent data storage, with SQLAlchemy as the ORM.

9.1 Database Tables

Users

Stores authentication and account information.

Field	Description
id	User identifier

username	Login username
password_hash	Hashed password
role	Retailer or Staff
retailer_id	Associated retailer reference

Retailers
Stores retailer information.

Field	Description
id	Retailer identifier
name	Retailer name
phone	Retailer phone number
credit_limit	Configured credit limit

Products
Stores product catalogue and stock information.

Field	Description
id	Product identifier
sku_code	Product SKU
name	Product name
dealer_price	Product dealer price
stock_cases	Available stock in cases
scheme_type	Configured scheme type
scheme_value	Configured scheme value

Orders
Stores order-level information.

Field	Description
id	Order identifier
retailer_id	Retailer reference
total_amount	Final order amount
status	Order status
created_at	Order creation date and time

Order Items
Stores individual products included in an order.

Field	Description
id	Order-item identifier
order_id	Order reference
product_id	Product reference
quantity	Ordered quantity
unit_price	Product unit price
free_quantity	Free quantity from schemes
subtotal	Calculated item subtotal

9.2 Database Relationships

- One retailer can place multiple orders.
- One order can contain multiple order items.
- Each order item references a product.
- A user account can be associated with a retailer.
- User roles determine permitted operations.

9.3 Database Configuration

The current SQLite database URL is:
sqlite:///./fmcg_orderhub.db

10. API Design

The backend exposes REST API endpoints through FastAPI.

Method	Endpoint	Purpose
GET	/	Application root response
GET	/health	Backend health check
POST	/auth/login	Authenticate a user
GET	/me	Retrieve authenticated account information
GET	/products	Retrieve product catalogue
GET	/retailers	Retrieve retailer information according to user role
POST	/orders	Create an order
GET	/orders	Retrieve orders according to user role
GET	/orders/export/tally	Export orders as CSV for staff

10.1 API Documentation

FastAPI provides interactive API documentation through Swagger UI.
Local documentation URL:
http://127.0.0.1:8000/docs

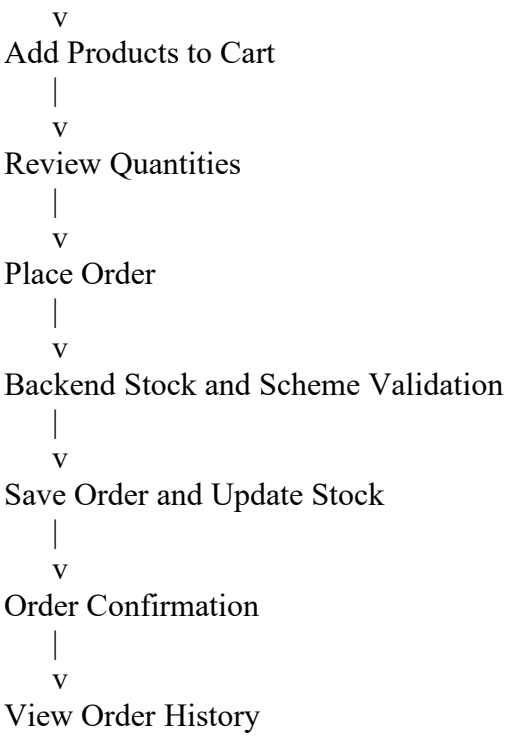
10.2 Access Control

- Retailers can access their own retailer information and orders.
- Staff can access retailer information and orders across the system.
- Only staff accounts can access the Tally CSV export endpoint.

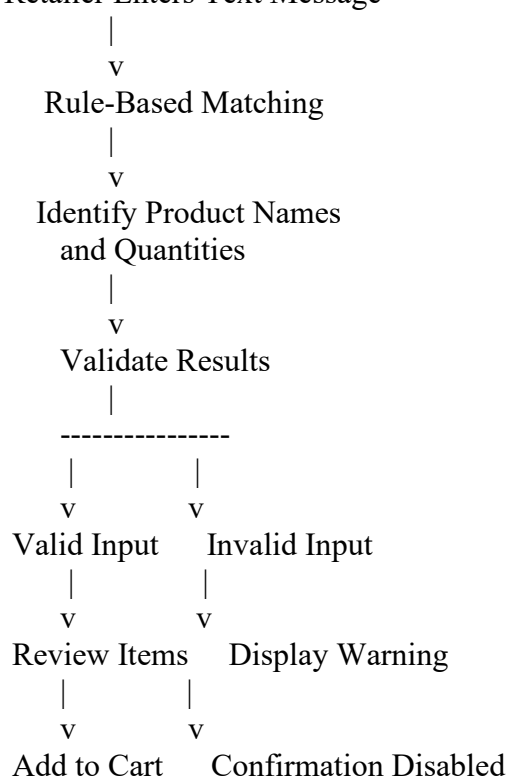
11. Main Application Flows

11.1 Retailer Login and Ordering Flow

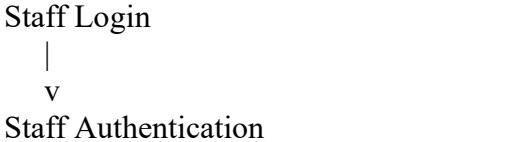
Retailer Login
    |
    v
Credential Validation
    |
    v
Retailer Dashboard
    |
    v
View Product Catalogue
    |



11.2 Text Order Capture Flow



11.3 Staff Order Management Flow



|
v
Staff Order Desk
|
v
View Orders from All Retailers
|
v
Review Order Details
|
v
Export Tally CSV

12. Security and Validation

The application includes the following security and validation measures:

- Password hashing using PBKDF2-HMAC-SHA256 with 310,000 iterations.
- HMAC-SHA256 signed authentication tokens.
- Eight-hour token expiration.
- Protected API endpoints.
- Role-based access control.
- Retailer-specific order access.
- Staff-only order export.
- Product stock validation.
- Quantity validation.
- Duplicate product validation.
- Backend-side pricing and scheme calculations.

The application reads its token signing secret from the JWT_SECRET_KEY environment variable and has a development-only fallback value for local use.

A strong secret must be configured before production deployment. The development fallback must not be used in production.

13. Error Handling

The application handles common errors, including:

- Invalid login credentials.
- Expired or missing authentication tokens.
- Empty shopping carts.
- Invalid product quantities.
- Duplicate products in an order.
- Insufficient product stock.
- Out-of-stock products.
- Unauthorized access.
- Failed order submissions.
- Failed CSV export requests.

Meaningful error responses and validation warnings help users understand and resolve common problems.

14. Testing and Verification

Testing was performed through automated backend tests and manual frontend verification.

14.1 Automated Backend Tests

The backend test suite contains 14 automated tests.

Test Case	Expected Result	Status
Home endpoint	Returns application response	Passed
Health endpoint	Returns health response	Passed
Valid retailer login	Authentication succeeds	Passed
Invalid login	Authentication is rejected	Passed
Products without login	Access is denied	Passed
Authenticated product access	Products are returned	Passed
Retailer-specific access	Retailer can view only their own store	Passed
Retailer CSV export	Export access is denied	Passed
Staff CSV export	CSV export succeeds	Passed
Free-product scheme	Free quantity is calculated correctly	Passed
Discount scheme	Applicable discount is calculated	Passed
Out-of-stock order	Order is rejected	Passed
Duplicate products	Duplicate product entries are rejected	Passed
Zero quantity	Invalid quantity is rejected	Passed

Automated test result: 14 passed, 3 warnings.

The warnings were non-fatal dependency and datetime deprecation warnings. No automated test failures were reported.

14.2 Manual Frontend Verification

The following workflows were checked manually during development.

Test Case	Expected Result	Status
Retailer login	Successful login	Passed
Staff login	Successful login	Passed
Product catalogue	Products and stock displayed	Passed
Add product to cart	Product added successfully	Passed
Single-product text capture	Supported product and quantity identified	Passed
Multiple-product text capture	Supported products identified	Passed
Unknown product text	Warning displayed and confirmation disabled	Passed
Missing quantity	Warning displayed and confirmation disabled	Passed
Out-of-stock product capture	Stock warning displayed	Passed
Place order	Order created successfully	Passed
Order history	Retailer order history displayed	Passed
Staff Order Desk	Orders from retailers displayed	Passed
Tally CSV export	CSV downloaded and verified	Passed

14.3 Frontend Build Verification

The following frontend checks completed successfully:

- ESLint validation using npm.cmd run lint.
- Production build using npm.cmd run build.

The production build completed successfully during development.

15. UI and Design Decisions

The application uses a dashboard-based layout with a consistent navigation area and content sections.

15.1 Navigation

The sidebar provides access to the main application sections. This keeps frequently used features accessible to retailers and staff.

15.2 Dashboard

The retailer dashboard provides an overview of the application and access to frequently used product and ordering functions.

15.3 Product Catalogue

Product cards display important product details such as product name, price, and stock information. This helps retailers review products before adding them to their cart.

15.4 Order Capture

The order capture interface provides a text input area and displays detected products and validation warnings. This allows retailers to review the results before confirming them.

15.5 Staff Order Desk

The staff screen provides a centralized order view and a CSV export action, reducing the need to check individual retailer accounts.

15.6 Design Approach

The interface uses a consistent dashboard layout, navigation, cards, tables, and status indicators to organize the main workflows.
The implemented screens are designed to make product browsing, order placement, text order capture, and staff order management easy to understand and use.

16. Limitations and Future Enhancements

16.1 Current Limitations

- Order capture uses predefined rule-based text matching rather than a trained AI model.
- Hindi voice-note transcription is not implemented.
- Image-based order extraction is not implemented.
- The current database uses SQLite and is intended for local project demonstration.
- CSV export is provided for Tally processing; direct Tally integration is not implemented.
- The project uses demonstration retailer and product data.
- The application has not yet been deployed to a production cloud environment.
- Production-scale performance for the business volume described in the brief has not been validated.
- GST inclusion or exclusion is not explicitly displayed in the current demonstration.

- Retailer credit-limit enforcement is not implemented in the current version.
- Mobile responsiveness has not been fully verified across different screen sizes.

16.2 Future Enhancements

- Integrate an AI model for flexible natural-language order understanding.
- Add Hindi voice-note transcription.
- Support image-based order extraction.
- Introduce live inventory synchronization.
- Integrate directly with Tally.
- Deploy the application using a production database and cloud infrastructure.
- Add order-status notifications.
- Add low-stock alerts.
- Improve monitoring, logging, and backup support.
- Perform load testing for higher order volumes.
- Verify and improve mobile responsiveness.

17. Conclusion

FMCG OrderHub provides a centralized digital ordering solution for an FMCG distributor. It enables retailers to browse products, place orders, review order history, and capture supported text orders using predefined matching rules. The backend validates stock, applies promotional schemes, and manages order records. Staff members can view orders from all retailers and export order information as CSV files for Tally processing. The project demonstrates a full-stack application using React, FastAPI, Python, SQLAlchemy, and SQLite, with authentication, role-based access control, order processing, and automated backend testing. The current implementation is a local demonstration and provides a foundation for future AI integration, additional capabilities, and production deployment.